

Model Overview

Peter Sørensen

Table of contents

1	Common Model: $y = Xb + e$	2
2	Main Interfaces	3
3	Glist as the Shared Data Carrier	3
4	Preparing Summary Statistics and Sparse LD	4
5	Three Axes for Choosing a Model	5
5.1	Single-Trait Versus Multi-Trait	5
5.2	Individual-Level Versus Summary Statistics	5
5.3	Plain Versus Annotation-Informed Priors	6
6	selection_s Support	6
7	Sparse Mixture Priors and Standardized Outputs	7
8	Posterior Summaries, Fine-Mapping, and Credible Sets	8
9	Practical Workflow Families	9
9.1	Direct BED Individual-Level ST-BLR	9
9.2	BED to Sufficient Statistics and Sparse LD	9
9.3	CSR Summary-Statistics ST-BLR	9
9.4	Annotation-Informed CSR ST-BLR	9
9.5	Fine-Mapping and Credible-Set Post-Processing	9
9.6	Simulation and Model Checking	9
10	Model and Function Map	10
11	Capability Matrix	12
12	Implemented Models and Research Extensions	13
13	What Is Not Guaranteed	14

sblr is an experimental R package for Bayesian linear regression of genomic marker effects. Its models estimate marker effects, posterior inclusion probabilities, variance components, and model-specific diagnostics from either individual-level PLINK BED data or summary statistics combined with linkage disequilibrium (LD).

The package is easiest to understand through four connected ideas:

1. **two main modelling interfaces:** `stblr_bed()` and `stblr_csr()`;
2. **a shared `Glist` data carrier** for genotype files, marker metadata, allele frequencies, and sparse-LD metadata;
3. **plain versus annotation-informed priors;** and
4. **standardized fit outputs** such as `bm`, `dm`, variance traces, component summaries, annotation summaries, and fine-mapping helpers.

This note maps those choices to the current public functions and to the notes that explain them in detail.

1 Common Model: $y = Xb + e$

For one trait, the common regression model is

$$y = Xb + e,$$

where X is an $n \times m$ genotype matrix, b is a vector of marker effects, and e is residual variation. Bayesian genomic regression combines this likelihood with a prior on marker effects and, often, latent inclusion or component indicators.

For n_t traits, the model becomes

$$Y = XB + E,$$

where each row of B is a marker-effect vector across traits. A genuinely joint multi-trait model can describe covariance among effects, residuals, or trait-sharing patterns. Running one single-trait model per phenotype can be useful and parallelizable, but it is not equivalent to a joint multi-trait likelihood.

The main posterior marker summaries are

$$\hat{b}_{it} = E(b_{it} \mid \text{data}) \quad \text{and} \quad \widehat{dm}_{it} = P(d_{it} = 1 \mid \text{data}),$$

represented in fitted objects by `bm` and `dm`. Depending on the backend, `dm` is interpreted as a posterior inclusion probability or as the posterior probability of being in a non-null component.

2 Main Interfaces

The two main user-facing modelling interfaces are:

- `stblr_bed()`: fits ST-BLR directly from individual-level PLINK BED markers referenced by `Glist`;
- `stblr_csr()`: fits annotation-unaware summary-statistics ST-BLR from sufficient statistics and disk-backed sparse LD stored in `Glist`.

Annotation-aware summary-statistics models are reached through the unified interface:

- `stblr_csr_annot()`: fits annotation-aware CSR models using `annotation_model` equal to "prior", "learned", "group", or "sbayesrc".

The older compatibility wrappers remain useful for explicit workflows:

- `stblr_csr_prior_annot()`;
- `stblr_csr_learn_annot()`;
- `stblr_csr_group_annot()`;
- `stblr_csr_sbayesrc_generic()`.

3 Glist as the Shared Data Carrier

`Glist` is the central object connecting BED and CSR workflows. It carries information about genotype files on disk, marker identifiers, chromosome/file structure, allele frequencies or MAF, and the marker order used by each backend. After sparse LD has been computed, `Glist` also carries the sparse-LD metadata needed by CSR models.

Common roles of `Glist` include:

- `Glist$bedfiles`: PLINK BED files used by individual-level and preparation workflows;
- `Glist$rsids`: marker identifiers in genotype/BED order;
- `Glist$rsidsLD`: marker identifiers in the sparse-LD reference order;
- `Glist$af` or `Glist$maf`: allele-frequency or MAF information used for scaling and `selection_s` models;
- `Glist$sparseLD`: sparse-LD metadata, including the disk-backed CSR prefix, marker subsets, LD-reference rows, and LD construction settings.

When marker-level information must be aligned to the LD order, use explicit matching rather than `%in%`. The current code pattern is

```
idx <- match(Glist$rsidsLD[[chr]], Glist$rsids[[chr]])
maf_ld <- Glist$maf[[chr]][idx]
```

or the corresponding allele-frequency field used by the workflow.

4 Preparing Summary Statistics and Sparse LD

The standard summary-statistics CSR workflow is:

```
stats <- make_stats(  
  Glist,  
  y,  
  nthreads = nthreads  
)  
  
Glist <- make_sparseLD(  
  Glist = Glist,  
  out_prefix = file.path(data_dir, "ld_test"),  
  pos_bp = NULL,  
  max_distance_bp = 0,  
  max_distance_variants = 1000,  
  r2_threshold = 0.001,  
  block_size = 1024,  
  nthreads = 4  
)
```

`make_stats()` computes genome-wide sufficient statistics such as marker-trait cross-products (`wy`), marker cross-products (`ww`), and phenotype sums of squares (`yy`) from BED-backed `Glist` data. `make_sparseLD()` computes a disk-backed sparse-LD CSR matrix and stores the resolved LD meta-data in `Glist$sparseLD`.

The updated `Glist` is then passed to CSR model fits:

```
fit <- stblr_csr(  
  stats = stats,  
  Glist = Glist,  
  method = "bayesC"  
)
```

Annotation-aware CSR models use the same `stats` and `Glist`, plus aligned annotation information:

```
fit_annot <- stblr_csr_annot(  
  stats = stats,  
  Glist = Glist,  
  annotations = annotations,  
  annotation_model = "sbayesrc"  
)
```

Direct individual-level fitting uses `Glist` and phenotypes without requiring a separate sparse-LD artifact:

```
fit_bed <- stblr_bed(
  Glist = Glist,
  y = y,
  method = "bayesC"
)
```

Check [Sparse-LD and BED workflows](#) for the current end-to-end preparation details.

5 Three Axes for Choosing a Model

5.1 Single-Trait Versus Multi-Trait

Single-trait BLR/ST-BLR estimates scalar effects and inclusion probabilities for one phenotype. Current BED, CSR, and annotation workflow examples are centered on this model family. Several ST-BLR wrappers accept a phenotype matrix or multiple trait-specific sufficient-statistic columns, but their marker-update structure should not automatically be interpreted as a full joint MTBLR sharing model.

Multi-trait BLR (MTBLR) treats marker effects as vectors and can model effect covariance, residual covariance, and explicit trait-sharing models. The public `sblr()` interface accepts summary-statistics inputs such as a trait cross-product matrix yy , marker-trait cross-products Xy , LD or marker cross-products XX , and optional multi-trait inclusion models. Full summary-statistics residual-covariance modeling additionally depends on cross-trait phenotype products and sample-overlap assumptions.

The current most polished end-to-end workflows are BED/CSR ST-BLR and annotation-informed CSR ST-BLR. Multi-trait theory remains central to the package and motivates the covariance outputs, simulation tools, and proposed annotation-aware extensions. Read [Multi-trait models](#) before interpreting trait sharing or covariance as a joint model result.

5.2 Individual-Level Versus Summary Statistics

With **individual-level data**, the sampler reads observed genotype columns from BED files and calculates marker-update quantities directly. The principal public interface is `stblr_bed()`.

With **summary statistics**, the sampler receives cross-products:

$$wy = X^\top y, \quad ww_j = x_j^\top x_j, \quad yy = y^\top y,$$

plus compatible LD and sample size n . The residual score can be maintained as

$$r = X^\top y - X^\top Xb.$$

`stblr_csr()` uses sufficient statistics from `make_stats()` with disk-backed sparse-LD metadata written into `Glist$sparseLD` by `make_sparseLD()`. The public `sblr()` interface represents the

broader summary-statistics model family and accepts dense or sparse marker cross-product structures through **XX**.

These representations, their information requirements, and sample-overlap issues are covered in [Data representations](#). The practical BED-to-CSR handoff is covered in [Sparse-LD and BED workflows](#).

5.3 Plain Versus Annotation-Informed Priors

A **plain/global prior** gives markers a common prior architecture, such as a global inclusion probability and marker-effect variance. A BayesC-style spike-and-slab representation is

$$d_{it} \sim \text{Bernoulli}(\pi_t), \quad b_{it} \mid d_{it} = 1 \sim N(0, v_{b,t}), \quad b_{it} \mid d_{it} = 0 = 0.$$

A BayesR-style prior replaces the binary inclusion state with mixture components, including a null component and several non-null variance multipliers.

An **annotation-informed prior** allows prior inclusion probabilities, variance scales, or mixture-component probabilities to vary by marker, marker group, or annotation profile. Current public CSR annotation models distinguish four implemented forms:

- fixed marker-specific inclusion probabilities or variance multipliers;
- learned annotation effects on inclusion or variance;
- grouped annotation architecture;
- SBayesRC-style annotation-dependent mixture probabilities.

These models use annotation information as a prior or evidence structure. They do not by themselves establish that an annotation is causal.

6 `selection_s` Support

CSR BayesC, CSR BayesR, and CSR SBayesRC support BayesS-style MAF-dependent prior variance scaling through the package argument `selection_s`. Effects in the CSR samplers are represented on the standardized-genotype scale, so the MAF-dependent prior scale is

$$q_j(S) = h_j^{S+1}, \quad h_j = 2p_j(1 - p_j),$$

where p_j is the minor allele frequency.

The current support matrix is:

Backend	Fixed global <code>selection_s</code>	Sampled trait-specific <code>selection_s</code>
<code>stblr_csr(method = "bayesC")</code>	yes	yes

Backend	Fixed global <code>selection_s</code>	Sampled trait-specific <code>selection_s</code>
<code>stblr_csr(method = "bayesR")</code>	yes	yes
<code>stblr_csr_annot(annotation_model = "sbayesrc")</code>	yes	yes
scheduled CSR BayesC	no	no
<code>stblr_csr_annot(annotation_model = "prior")</code>	no	no
<code>stblr_csr_annot(annotation_model = "learned")</code>	no	no
<code>stblr_csr_annot(annotation_model = "group")</code>	no	no
BED backends	no	no

Current sampled-`selection_s` defaults are:

```
selection_s_init = 0
selection_s_prior = c(-3, 2)
selection_s_proposal_sd = 0.35
```

For SBayesRC, annotations affect component probabilities, while `selection_s` affects marker-specific effect-size prior variances. See [selection_s implementation status](#) for the full support matrix, scale convention, defaults, and Metropolis-Hastings kernels.

7 Sparse Mixture Priors and Standardized Outputs

Sparse Bayesian regression allows many markers to have exactly zero current effects while a subset belongs to active components. The inclusion or component state records whether marker i is active for trait t . Across MCMC samples:

- `dm[i, t]` estimates the posterior inclusion probability or posterior non-null component probability;
- `bm[i, t]` estimates the posterior mean marker effect;
- `vbs`, `vgs`, and `ves` record marker-effect, genetic, and residual variance traces;
- `vle` and `vld` record linkage-equilibrium and LD variance decomposition traces where available;
- `input` stores model, backend, data, and sampler metadata;
- `chains` stores optional compact chain-level summaries when `keep_chains = TRUE`.

BayesR and SBayesRC fits also return component summaries:

- `comp_prob`: marker-by-component posterior probabilities by trait;
- `dm_component_mean`: posterior mean component/effect-class summary.

The null component naming differs by model:

- BayesR: `component_0`, with `dm = 1 - P(component_0)`;
- SBayesRC: `gamma_0.00`, with `dm = 1 - P(gamma_0.00)`.

Annotation-aware fits may return annotation summaries such as `annotation_summary`, `annotation_pi`, `annotation_effects`, `annotation_prior`, `alpha`, and `sigmaSqAlpha`, depending on the backend.

Sampled-selection_s fits return:

- `selection_s`: posterior mean by trait;
- `selection_s_sd`, `selection_s_min`, and `selection_s_max`;
- `selection_s_trace`: iteration-by-trait trace matrix;
- `selection_s_acceptance`: Metropolis-Hastings acceptance rate by trait.

With `keep_chains = TRUE`, chain-level sampled-S outputs are stored in `fit$chains[[trait]][[chain]]$selection_s` and `fit$chains[[trait]][[chain]]$selection_s_acceptance` where supported.

8 Posterior Summaries, Fine-Mapping, and Credible Sets

The core samplers return posterior summaries and traces. Several R helpers turn those outputs into model diagnostics, plots, fine-mapping summaries, and credible sets.

Common helper functions include:

- `summarise_stblr_posterior()`: summarizes posterior traces such as `vbs`, `vgs`, `ves`, `vle`, `vld`, and sampled-selection_s traces where present;
- `plot_stblr_posterior()`: plots posterior summaries and trace diagnostics;
- `summarise_stblr_bayesr_components()`: summarizes BayesR/SBayesRC component probability outputs;
- `summarise_stblr_maf_architecture()`: provides post-hoc descriptive summaries of MAF-dependent architecture, separate from sampler-level `selection_s`;
- `make_stblr_credible_sets()`: constructs credible sets from posterior marker probabilities and marker/LD information;
- `check_stblr_backend_consistency()`: checks that fit objects contain the expected backend-specific fields.

Fine-mapping diagnostics are primarily represented through marker PIPs in `fit$dm`, optional LD-swap summaries in `fit$ld_swap`, and helper-level post-processing such as credible-set construction. Credible sets are therefore not the core sampler return object; they are constructed from fitted model outputs.

LD-swap output is available where `updateLDswap = TRUE` is supported. The regular CSR backends use LD-swap diagnostics for fine-mapping and high-LD regions. Scheduled CSR and BED backends should not be assumed to provide the same LD-swap output unless the specific backend documentation says so.

9 Practical Workflow Families

The package’s current workflows can be read as six connected families.

9.1 Direct BED Individual-Level ST-BLR

Use `stblr_bed()` when BED genotypes and aligned phenotypes remain available during fitting. Genotypes are read in blocks, and a separate LD artifact is not required.

9.2 BED to Sufficient Statistics and Sparse LD

Use `make_stats()` to derive `wy`, `ww`, and `yy` from a BED-backed `Glist` and phenotypes. Use `make_sparseLD()` to derive disk-backed sparse LD from the same markers, samples, allele frequencies, and scaling convention. This preparation workflow creates reusable inputs for CSR models.

9.3 CSR Summary-Statistics ST-BLR

Use `stblr_csr()` with the sufficient-statistics object and sparse-LD metadata stored in `Glist`. This path is appropriate when the prepared cross-products and LD files should be reused without repeated BED access during fitting.

9.4 Annotation-Informed CSR ST-BLR

Start from the same `stats` and sparse-LD metadata in `Glist`, then add marker-aligned annotation information or marker-specific priors. Use `stblr_csr_annot()` to choose among fixed-prior, learned-annotation, group-annotation, and SBayesRC-style annotation models.

9.5 Fine-Mapping and Credible-Set Post-Processing

Use marker PIPs, optional LD-swap diagnostics, local fine-mapping helpers, and credible-set helpers to move from global model fits to regional or signal-level summaries. These are post-processing or local-refit steps layered on top of fitted model objects.

9.6 Simulation and Model Checking

Use `mtsim()` to simulate multi-trait phenotypes, marker effects, and known shared or trait-specific signals. Use `mtsim_annotation()` to additionally simulate overlapping annotations, causal enrichment, and marker-specific prior values. Simulation outputs provide known truth for checking marker recovery, annotation summaries, model behavior, and BED-versus-CSR preparation.

Simulation is a model-checking tool, not evidence that the same assumptions hold in real data.

10 Model and Function Map

Function	Primary role	Data representation	Trait/prior emphasis	Read next
<code>stblr_bed()</code>	Fit ST-BLR directly from PLINK BED markers and phenotypes.	Individual-level BED through <code>Glist</code> .	Plain/global BayesC or BayesR ST-BLR; accepts one or more phenotype columns; no sampler-level <code>selection_s</code> .	ST-BLR interface and output map , Sparse-LD and BED workflows , Single-trait models
<code>stblr_bed_marker()</code>	Lower-level direct BED marker interface retained for compatibility and specialized workflows.	Individual-level BED.	Plain/global ST-BLR.	Sparse-LD and BED workflows
<code>make_stats()</code>	Derive marker-trait and phenotype sufficient statistics from BED-backed <code>Glist</code> data.	BED to summary statistics.	Preparation, not a fitted model.	Data representations , ST-BLR interface and output map
<code>make_sparseLD()</code>	Construct disk-backed sparse LD from BED-backed <code>Glist</code> data and store metadata in <code>Glist\$sparseLD</code> .	BED to CSR LD.	Preparation, not a fitted model.	Sparse-LD and BED workflows , Technical sparse LD and CSR , ST-BLR interface and output map
<code>stblr_csr()</code>	Fit annotation-unaware ST-BLR from <code>stats</code> and sparse LD in <code>Glist</code> .	Summary statistics plus disk-backed sparse LD.	CSR BayesC or BayesR, with fixed global or sampled trait-specific <code>selection_s</code> where supported.	Single-trait models , selection_s status , ST-BLR interface and output map

Function	Primary role	Data representation	Trait/prior emphasis	Read next
<code>stblr_csr_annot()</code>	Unified annotation-aware CSR interface.	Summary statistics plus CSR LD and annotations.	<code>annotation_model = "prior", "learned", "group", or "sbayesrc"</code> ; only SBayesRC supports fixed and sampled <code>selection_s</code> .	Annotation-informed models, ST-BLR interface and output map
<code>stblr_csr_prior_annot()</code>	Compatibility wrapper for fixed marker-specific priors, supplied directly or derived from fixed annotation coefficients.	Summary statistics plus CSR LD and optional annotations.	Fixed annotation-informed prior; no sampler-level <code>selection_s</code> .	Annotation-informed models
<code>stblr_csr_learn_annot()</code>	Compatibility wrapper for learned annotation effects on marker inclusion and optionally effect-variance multipliers.	Summary statistics plus CSR LD and annotations.	Direct learned annotation prior; no sampler-level <code>selection_s</code> .	Annotation-informed models, Technical annotation priors
<code>stblr_csr_group_annot()</code>	Compatibility wrapper for group-level inclusion probabilities and variance multipliers.	Summary statistics plus CSR LD and marker groups.	Group prior; no sampler-level <code>selection_s</code> .	Annotation-informed models
<code>stblr_csr_sbayesrc_annot()</code>	Compatibility wrapper for SBayesRC-style annotation-dependent BayesR prior with marker-specific mixture probabilities.	Summary statistics plus CSR LD and overlapping annotations.	SBayesRC mixture prior; supports fixed and sampled <code>selection_s</code> .	Annotation-informed models, Technical SBayesRC-style annotations

Function	Primary role	Data representation	Trait/prior emphasis	Read next
<code>sblr()</code>	Fit the broader summary-statistics BLR interface, including multi-trait inputs and optional sharing models.	Summary statistics plus marker cross-products/LD.	Plain single- or multi-trait model components.	Multi-trait models
<code>finemap_stblr_csr()</code>	Local fine-mapping/refitting around predefined marker regions using architecture from a global CSR fit.	Global CSR fit plus <code>Glist</code> , <code>stats</code> , and marker sets.	Local regional summaries and optional credible sets.	Workflows and outputs
<code>make_stblr_credib()</code>	Construct credible sets from fitted posterior marker probabilities and marker/LD information.	Fit summaries and marker/LD metadata.	Post-processing helper, not a core sampler.	Workflows and outputs
<code>mtsim()</code>	Simulate multi-trait phenotypes and marker effects with shared and trait-specific signals.	Simulated matrix or <code>Glist</code> genotypes.	Model checking and multi-trait truth.	Workflows and outputs
<code>mtsim_annotation()</code>	Simulate multi-trait phenotypes with overlapping annotation enrichment and marker-specific priors.	Simulated matrix or <code>Glist</code> genotypes plus annotations.	Annotation-model checking.	Annotation-informed models, Workflows and outputs

11 Capability Matrix

Model/API	Fixed selection_s	Sampled selection_s	Annotation- aware	Component probabili- ties	Annotation outputs	LD- swap/fine- mapping diagnostics
stblr_csr(method = "bayesC")	yes	yes	no	no	no	yes, regular CSR where enabled
stblr_csr(method = "bayesR")	yes	yes	no	yes	no	yes, regular CSR where enabled
stblr_bed(method = "bayesC")	no	no	no	no	no	no core LD-swap output
stblr_bed(method = "bayesR")	no	no	no	yes	no	no core LD-swap output
stblr_csr_annot(annotation_model = "prior")			yes	no	yes	backend-specific diagnostics only
stblr_csr_annot(annotation_model = "learned")			yes	no	yes	backend-specific diagnostics only
stblr_csr_annot(annotation_model = "group")			yes	no	yes	backend-specific diagnostics only
stblr_csr_annot(annotation_model = "sbayesrc")			yes	yes	yes	yes, where enabled

12 Implemented Models and Research Extensions

The model/function map above describes current public interfaces. The archived technical drafts also contain deeper or proposed directions, including:

- empirical-Bayes loops that use first-stage marker summaries to estimate annotation-level priors for later genomic fits;
- direct annotation-aware MTBLR sharing-model priors and covariance models;
- sparse annotation representations and sampler implementation sketches;
- warm-started regional MTBLR workflows;
- reconstruction of cross-trait SSY and overlap matrices for joint summary-statistics models.

These ideas should be preserved because they explain the intended model landscape, but they must be labeled as implemented, experimental, or proposed when moved into maintained notes. Public

function availability alone also does not guarantee that every model path has an equally polished end-to-end workflow.

13 What Is Not Guaranteed

- **BED and CSR fits are not exactly equal.** Under matched markers, rows, scaling, priors, and MCMC settings they should agree qualitatively, but sparse-LD approximation, update scheduling, floating-point precision, and Monte Carlo variation prevent exact equality.
- **CSR disk internals are not a manual file-format API.** Use package writers, readers, and model wrappers rather than editing row-pointer, column-index, value, or metadata files directly.
- **Annotation effects are not causal proof.** Enrichment, learned prior coefficients, posterior inclusion, and shared genetic signals can generate hypotheses but do not identify causal biological mechanisms by themselves.
- **Short workflow chains are not final inference.** Demonstration settings establish object handoffs and model execution. Real analyses require longer chains, convergence assessment, sensitivity analysis, and posterior predictive checks.

14 How to Read These Notes

Choose the next note according to the question:

- To understand the current package interfaces, data flow, model capabilities, standardized outputs, and helper functions, read [ST-BLR interface and output map](#).
- To understand what BED, sufficient statistics, sparse LD, yy, and SSY represent, read [Data representations](#).
- To prepare aligned BED-derived statistics and disk-backed CSR LD, read [Sparse-LD and BED workflows](#).
- To understand plain BED and CSR ST-BLR models, read [Single-trait models](#).
- To understand joint effects, covariance, trait sharing, and sample overlap, read [Multi-trait models](#).
- To choose among fixed, group, learned, empirical-Bayes, and SBayesRC-style priors, read [Annotation-informed models](#).
- To run workflows and interpret compact fit summaries, read [Workflows and outputs](#).
- To understand fixed and sampled `selection_s` support, read [selection_s implementation status](#).

For full derivations and implementation-sensitive contracts, use the technical appendices:

- [Technical summary statistics](#)
- [Technical multi-trait sample overlap](#)
- [Technical sparse LD and CSR](#)
- [Technical annotation priors](#)
- [Technical SBayesRC-style annotations](#)